

Chapitre 2 : Algèbre de Boole

Introduction

Les circuits logiques constituent une partie très importante dans les systèmes électroniques. Nous distinguons deux types de circuits logiques dont le fonctionnement est différent : les circuits combinatoires et les circuits séquentiels. Les premiers expriment simplement leurs sorties à partir de leurs entrées ; les seconds font dépendre leurs sorties de leurs entrées mais également des sorties précédentes.

Ces circuits logiques peuvent prendre deux valeurs, 0 ou 1, correspondant à deux niveaux de tension. Classiquement, le 0 est associé au niveau bas de la tension, proche de 0 V, alors que le 1 est associé au niveau haut, historiquement de 5 V, mais qui a diminué et est maintenant proche de 1 V.

Cette information circule à travers les différents circuits de l'ordinateur, travaillant en logique numérique, qui sont modélisés par l'algèbre de Boole.

Tous les circuits électroniques sont composés de transistors, de l'ordre du milliard dans les processeurs actuels. Comme il est impossible d'étudier leur comportement individuel, ils ont été modélisés et regroupés sous forme de circuits élémentaires que nous allons présenter. Tous les circuits intégrés sont composés, sur une grande échelle, de ces circuits élémentaires.

1. Algèbre de Boole

L'analyse des circuits logiques nécessite l'utilisation d'une algèbre spécifique, dite « booléenne », fruit des travaux du mathématicien anglais du XIX^e siècle George Boole. Ces travaux ont défini un ensemble d'opérateurs de base, ainsi que leurs propriétés, qui composent une algèbre permettant de concevoir tout type de circuit. La construction de fonctions logiques répondant à des contraintes précises et leur représentation sous forme de circuits électroniques se font à l'aide d'opérateurs élémentaires.

1.1. Opérateurs de base de l'algèbre de Boole

1.1.1. Opérateur ET

L'opérateur ET correspond à la multiplication logique, le résultat s'exprime ainsi :

La liste complète des valeurs que peut prendre la fonction booléenne S en fonction de toutes les combinaisons possibles des valeurs A et B peuvent être résumées

A	B	S = A.B
0	0	
0	1	
1	0	
1	1	

Tableau 1 : table de vérité de l'opérateur ET

1.1.2. Opérateur OU

L'addition logique entre A et B donne :

La table de vérité de l'opérateur OU est représentée par le tableau 2.

A	B	$S = A+B$
0	0	
0	1	
1	0	
1	1	

Tableau 2 : table de vérité de l'opérateur OU

1.1.3. Opérateur inverseur

L'opérateur inverseur à une seule variable en entrée. Le résultat S de cet opérateur sur une variable booléenne A donne son complément. On note :

La table de vérité de l'opérateur inverseur est représentée par le tableau 3.

A	$S = \overline{A}$
0	
1	

Tableau 2 : table de vérité de l'opérateur inverseur

1.2. Théorèmes de l'algèbre de Boole

Par la suite, nous présentons l'ensemble des théorèmes qui régissent l'algèbre de Boole.

1.2.1. Théorème de commutativité

1.2.2. Théorème d'associativité

1.2.3. Théorème de distributivité

1.2.4. Théorème d'idempotence

1.2.5. Théorème de complémentation

1.2.6. Théorème des constantes

1.2.7. Théorème d'absorption

1.2.8. Théorème d'allégement

1.2.9. Théorème De Morgan

1.3.Représentation d'une fonction logique

Une fonction booléenne est entièrement définie par sa table de vérité mais cela n'est pas d'une grande aide pour implémenter cette fonction sous forme de circuit combinatoire. En effet, les seuls circuits élémentaires à disposition sont les portes logiques et celles-ci reproduisent les opérateurs de base (NON, OU, ET...) qui sont dans l'expression algébrique d'une fonction. Il faut donc transformer la table de vérité de la fonction en expression algébrique afin de pouvoir la calculer et surtout de pouvoir physiquement l'implémenter à l'aide des portes logiques.

1.3.1. Représentation algébrique

Une fonction logique écrite sous forme algébrique peut être représentée sous forme de somme, produit, somme canonique ou produit canonique.

1.3.1.1. Forme somme

Exemples :

1.3.1.2. Forme produit

.

Exemples :

Remarque : d'après le théorème de Morgan, si une expression est écrite sous la forme somme, son inverse sera sous la forme produit (et vice-versa), comme le montre l'exemple suivant :

1.3.1.3. Forme somme canonique

Ces termes se désignent sous le nom **mintermes**.

Exemples :

Soient les fonctions à trois variables A, B, C :

Les deux premières expressions sont écrites sous la forme de somme canonique sauf la fonction S_3 , car son premier terme n'est pas un minterme (vu que C n'apparaît pas dans ce terme).

1.3.1.4. Forme produit canonique

Ces termes se désignent sous le nom **maxtermes**.

Exemples :

Soient les fonctions à quatre variables A, B, C, D :

S_2 n'est pas sous forme de produit canonique car le premier produit ne contient pas les variables C et D donc ce n'est pas un maxterme.

Remarque : d'après le théorème de Morgan, si une expression est écrite sous la forme somme canonique, son inverse sera sous la forme produit canonique (et vice-versa), comme le montre l'exemple suivant :

1.3.2. Table de vérité d'une fonction logique

Une expression logique $S(A, B, C, \dots)$, fonction des variables A, B, C, ..., peut être représentée par une table de vérité. Cette table donne les valeurs que peut prendre S, suivant les différentes combinaisons des variables A, B, C, ...

Exemple : soit la table de vérité suivante à trois variables : A, B, C :

A	B	C	S
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

On note qu'une table de vérité donne l'expression S sous forme de somme canonique.

1.4. Simplification des expressions logiques

La simplification d'une expression logique, consiste à réduire cette expression à sa forme la plus simple mais équivalente, c'est-à-dire à un nombre minimal de terme et un nombre minimal de variable dans chaque terme.

1.4.1. Simplification algébrique

Les théorèmes de Boole étudiés précédemment peuvent nous être utiles pour simplifier une expression logique.

Exemple 1 :

Exemple 2 :

Exemple 3 :

1.4.2. Simplification par diagramme de Karnaugh

Pour une fonction de deux ou trois variables, il est simple d'utiliser la table de vérité pour exprimer et ensuite simplifier l'expression algébrique de la fonction. Si l'on a une fonction de quatre ou cinq variables, il existe une méthode graphique permettant d'arriver au même résultat : le tableau de Karnaugh.

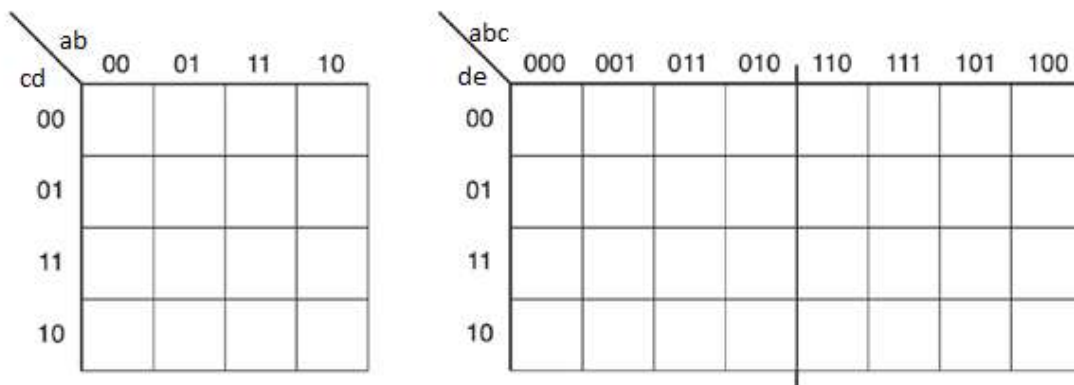


Figure 1 : Tableaux de Karnaugh pour quatre et cinq variables

1.4.2.1. Construction du diagramme de Karnaugh

Exemples :

a- Donner les diagrammes de Karnaugh relatifs aux tables de vérité suivant :

A	B	S
0	0	1
0	1	0
1	0	0
1	1	1



A	B	C	S
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0



b- Donner les diagrammes de Karnaugh relatifs aux fonctions logiques suivantes :

$$S_1 = A.\bar{B}.C.D + A.\bar{B}.C.\bar{D} + A.B.C.\bar{D}$$

$$S_2 = (\bar{A} + B + \bar{C}) . (\bar{A} + \bar{B} + \bar{C}) . (A + B + \bar{C})$$

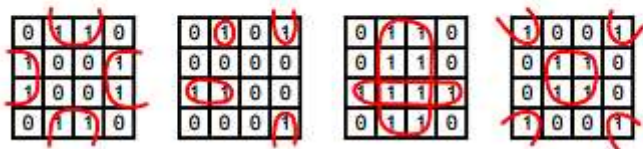
Comme cette fonction n'est pas écrite sous forme de somme canonique. On utilisera son inverse pour déterminer le diagramme de Karnaugh.

1.4.2.2. Simplification par le diagramme de Karnaugh

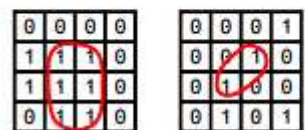
La méthode de simplification par diagramme de karnaugh est la suivante :

Exemples de groupements

POSSIBLES



IMPOSSIBLES



Le principe de regroupements est le suivant :

-
-
-
-
-

Exemples :

	$\bar{A}$	A
$\bar{B}$	1	0
B	0	1

	$\bar{A}$	A
$\bar{B}$	1	1
B	0	1

	$\bar{A}.\bar{B}$	$\bar{A}.B$	$A.B$	$A.\bar{B}$
$\bar{C}$	0	1	1	0
C	0	0	0	0

	$\bar{A}.\bar{B}$	$\bar{A}.B$	$A.B$	$A.\bar{B}$
$\bar{C}$	1	0	0	1
C	0	0	0	0

	$\bar{A}.\bar{B}$	$\bar{A}.B$	$A.B$	$A.\bar{B}$
$\bar{C}$	0	0	0	0
C	1	1	1	1

	$\bar{A}.\bar{B}$	$\bar{A}.B$	$A.B$	$A.\bar{B}$
$\bar{C}$	1	1	0	0
C	1	1	0	1

	$\bar{A}.\bar{B}$	$\bar{A}.B$	$A.B$	$A.\bar{B}$
$\bar{C}.\bar{D}$	0	0	0	1
$\bar{C}.D$	0	1	1	0
$C.D$	0	1	1	0
$C.\bar{D}$	0	0	1	0

	$\bar{A}.\bar{B}$	$\bar{A}.B$	$A.B$	$A.\bar{B}$
$\bar{C}.\bar{D}$	1	1	1	1
$\bar{C}.D$	0	0	0	0
$C.D$	0	0	0	1
$C.\bar{D}$	1	1	1	1

	$\bar{A}.\bar{B}$	$\bar{A}.B$	$A.B$	$A.\bar{B}$
$\bar{C}.\bar{D}$	1	0	0	1
$\bar{C}.D$	0	0	0	0
$C.D$	0	0	0	0
$C.\bar{D}$	1	0	0	1

	$\overline{A}.\overline{B}$	$\overline{A}.B$	$A.B$	$A.\overline{B}$	1GLSI
$\overline{C}.\overline{D}$	1	1	0	0	
$\overline{C}.D$	1	1	0	0	
$C.D$	1	1	0	0	
$C.\overline{D}$	1	1	0	0	

Remarque 1 :

- Un groupement de 1 case n'élimine aucune variable,
- Un groupement de 2 cases élimine 1 variable,
- Un groupement de 4 cases élimine 2 variables,
- Un groupement de 8 cases élimine 3 variables.

Remarque 2 :

- La simplification peut ne pas être unique

Dans l'exemple suivant, le regroupement des doublets suivants peut se faire de deux manières différentes :

	$\overline{A}.\overline{B}$	$\overline{A}.B$	$A.B$	$A.\overline{B}$
$\overline{C}$	1	1	1	0
C	1	0	1	1

	$\overline{A}.\overline{B}$	$\overline{A}.B$	$A.B$	$A.\overline{B}$
$\overline{C}$	1	1	1	0
C	1	0	1	1

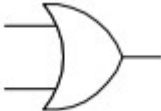
- Présence d'états indifférents

Dans certains problèmes, il n'est pas important qu'une ou plusieurs combinaisons de sortie de la table de vérité soit à 0 ou à 1, il s'agit alors d'états indifférents.

	$\overline{A}.\overline{B}$	$\overline{A}.B$	$A.B$	$A.\overline{B}$
$\overline{C}$	1	0	x	x
C	1	0	0	1

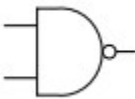
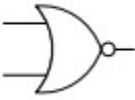
2. Portes logiques

2.1. Portes logiques élémentaires

Fonction logique	Symbole	Table de vérité															
NON (NOT) $S = \overline{A}$		<table><tr><td>A</td><td>$S = \overline{A}$</td></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	$S = \overline{A}$	0	1	1	0									
A	$S = \overline{A}$																
0	1																
1	0																
Ou (OR) $S = A + B$		<table><tr><td>A</td><td>B</td><td>$S = A+B$</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	$S = A+B$	0	0	0	0	1	1	1	0	1	1	1	1
A	B	$S = A+B$															
0	0	0															
0	1	1															
1	0	1															
1	1	1															
ET (AND) $S = A . B$		<table><tr><td>A</td><td>B</td><td>$S = A.B$</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr></table>	A	B	$S = A.B$	0	0	0	0	1	0						
A	B	$S = A.B$															
0	0	0															
0	1	0															

			1	0	0	
			1	1	1	

2.2. Portes logiques complètes

Fonction logique	Symbole	Table de vérité															
NON-ET (NAND) $S = \overline{A \cdot B}$		<table> <tr> <th>A</th><th>B</th><th>$S = \overline{A \cdot B}$</th></tr> <tr> <td>0</td><td>0</td><td>1</td></tr> <tr> <td>0</td><td>1</td><td>1</td></tr> <tr> <td>1</td><td>0</td><td>1</td></tr> <tr> <td>1</td><td>1</td><td>0</td></tr> </table>	A	B	$S = \overline{A \cdot B}$	0	0	1	0	1	1	1	0	1	1	1	0
A	B	$S = \overline{A \cdot B}$															
0	0	1															
0	1	1															
1	0	1															
1	1	0															
NON-Ou (NOR) $S = \overline{A + B}$		<table> <tr> <th>A</th><th>B</th><th>$S = \overline{A + B}$</th></tr> <tr> <td>0</td><td>0</td><td>1</td></tr> <tr> <td>0</td><td>1</td><td>0</td></tr> <tr> <td>1</td><td>0</td><td>0</td></tr> <tr> <td>1</td><td>1</td><td>0</td></tr> </table>	A	B	$S = \overline{A + B}$	0	0	1	0	1	0	1	0	0	1	1	0
A	B	$S = \overline{A + B}$															
0	0	1															
0	1	0															
1	0	0															
1	1	0															

Remarque : universalité des portes NAND et NOR

Les portes NAND et NOR sont utilisées pour générer n'importe quelle fonction booléenne. On dit qu'elles sont des portes complètes ou universelles.

Pour réaliser le circuit logique d'une fonction logique S quelconque à partir d'un seul type de portes soit NAND soit NOR, on doit appliquer une double inversion puis le théorème de De Morgan à l'expression de S de manière à retrouver l'expression appropriée. On peut appliquer autant de double inversions qu'il est nécessaire.

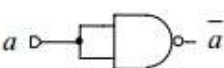
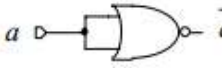
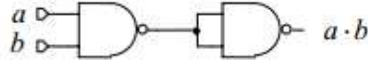
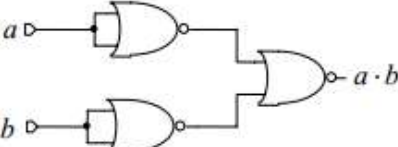
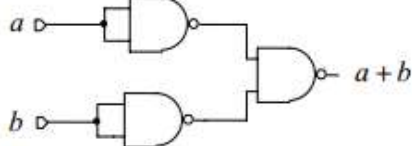
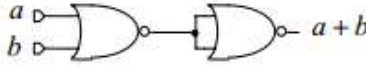
Exemple :

On utilisant uniquement des portes NAND puis NOR, élaborer le circuit relatif à l'expression suivante : $S = \overline{A} \cdot B + A \cdot \overline{B}$

a- Utilisation des portes NAND

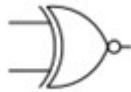
b- Utilisation des portes NOR

Le tableau suivant représente les circuits équivalents de portes logiques réalisées à partir des portes NAND et NOR

Portes	Circuits équivalent avec des portes NAND	Circuits équivalent avec des portes NOR
NOT 		
AND 		
OR 		

2.3. Porte ou-exclusif (XOR), ni-exclusif (XNOR)

Fonction logique	Symbole	Table de vérité												
OU-exclusif (XOR) $S = A \oplus B = \bar{A} \cdot B + A \cdot \bar{B}$		<table border="1"> <tr> <th>A</th><th>B</th><th>$S = A \oplus B$</th></tr> <tr> <td>0</td><td>0</td><td>0</td></tr> <tr> <td>0</td><td>1</td><td>1</td></tr> <tr> <td>1</td><td>0</td><td>1</td></tr> </table>	A	B	$S = A \oplus B$	0	0	0	0	1	1	1	0	1
A	B	$S = A \oplus B$												
0	0	0												
0	1	1												
1	0	1												

		1	1	0																
NON-Ou-exclusif (XNOR) $S = \overline{A \oplus B} = A.B + \overline{A}.\overline{B}$		<table><tr><th>A</th><th>B</th><th>$S = \overline{A \oplus B}$</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>				A	B	$S = \overline{A \oplus B}$	0	0	1	0	1	0	1	0	0	1	1	1
A	B	$S = \overline{A \oplus B}$																		
0	0	1																		
0	1	0																		
1	0	0																		
1	1	1																		

3. Synthèse de circuits logiques combinatoires

3.1. Etapes de synthèse

La synthèse d'un circuit logique relatif à un problème, doit suivre les étapes suivantes :

-
-
-
-
-

3.2. Exemples de synthèses de circuits logiques

Exemple 1 :

Déterminer le circuit logique à trois entrées A, B et C de façon que la sortie S soit à 1 quand au moins deux entrées sont à l'état Haut.

Résultat :

La table de vérité est la suivante :

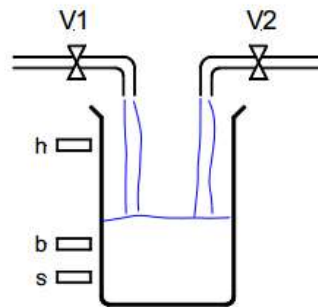
Cette expression peut être simplifiée à l'aide du diagramme de Karnaugh suivant :

	$\overline{A}.\overline{B}$	$\overline{A}.B$	$A.B$	$A.\overline{B}$
$\overline{C}$				
C				

Le circuit électronique qui représente cette expression logique est le suivant :

Exemple 2 :

Un réservoir est alimenté par deux vannes V1 et V2. On distingue trois niveaux : Sécurité (s), Bas (b), et Haut (h).



Lorsque le niveau est au dessous de (s), on ouvre les vannes V1 & V2

Lorsque le niveau est entre (s) et (b) on ouvre la vanne V1 seulement

Lorsque le niveau est entre (b) et (h) on ouvre la vanne V2 seulement

Lorsque le niveau est détecté par (h) on arrête les deux vannes

QUESTION :

Déterminer les équations logiques d'ouverture des deux vannes V1 et V2 en fonction du niveau détecté par les 3 capteurs (s), (b), (h).

Nota : Lorsque le niveau est détecté par un capteur, son état = 1